

In The Name of God



Navid Zare (40435071)

Ali Ahmadi (40464248)

Komeil Khodayar

Digital Image Processing

Assignment 1: Point Operations, Histogram Processing, and Pixel Resolution

1. Introduction

1.1 Background and Problem Context

Digital image processing encompasses a broad class of computational techniques applied to image data in order to extract meaningful information, enhance visual quality, or prepare imagery for subsequent analysis. Among the most fundamental of these techniques are spatial domain operations—methods that manipulate pixel intensity values directly without transforming the image into a frequency domain.

This report documents the complete implementation and experimental evaluation of four categories of such operations:

1. Pixel-resolution resampling
2. Intensity point operations
3. Histogram-based contrast enhancement
4. Histogram matching with color-space manipulation

1.2 Research Objectives

This homework pursues five principal objectives:

- (1) Section 0: Demonstrate proficiency in two-dimensional array indexing by embedding a small matrix into a larger matrix via a generalized diamond (rotated checkerboard) pattern, deriving the mathematical coordinate mapping analytically.
- (2) Section 1: Investigate the effect of spatial downsampling on image quality and compare three upsampling interpolation kernels quantitatively via Mean Squared Error (MSE) and Peak Signal-to-Noise Ratio (PSNR):
 - Nearest-Neighbor
 - Bilinear
 - bicubic
- (3) Section 2: Implement and visualize four classical point-operation intensity transformations
 - Image negative
 - Logarithmic
 - Gamma
 - Piecewise-linear contrast stretching

In addition, we also perform bit-plane decomposition to analyze the information content of individual intensity bits.

- (4) Section 3: Derive histogram equalization from first principles at both the global scale (Global Histogram Equalization, GHE) and the local scale (Local Histogram Equalization, LHE) using a sliding-window Neighborhood, and critically compare their respective trade-offs.
- (5) Section 4: Implement histogram specification (matching) to transfer the statistical distribution of one image to another, and develop mathematically rigorous $\text{RGB} \rightarrow \text{HSV}$, $\text{HSV} \rightarrow \text{RGB}$ conversions from scratch to apply equalization exclusively to the luminance channel while preserving color consistency.

2. Methodology

2.1 Section 0: Diamond-Pattern Matrix Embedding (Bonus)

The objective is to embed a smaller $N \times M$ matrix S into a larger $H \times W$ matrix L such that the elements of S appear along the diagonals of a rotated 45° diamond pattern. The mapping is derived by observing that a 45° clockwise rotation applied to grid coordinates $(i, j) \in S$ yields:

$$\begin{aligned}\text{row}(i, j) &= (M - 1) - j + i + \text{row_offset} \\ \text{col}(i, j) &= i + j + \text{col_offset}\end{aligned}$$

where the offsets center the diamond within L . The diamond requires a minimum of $(N + M - 1)$ rows and $(N + M - 1)$ columns; if L does not satisfy this constraint, the function returns the string "Impossible".

2.2 Section 1: Pixel Resolution and Interpolation

2.2.1 Downsampling

Resolution reduction is performed by direct pixel subsampling: given a downsampling factor N , the output image I_{down} is formed by retaining every N -th pixel in each spatial dimension:

$$I_{\text{down}}[i, j] = I_{\text{orig}}[i \cdot N, j \cdot N]$$

Factors $N \in \{2, 4, 8\}$ are evaluated. Because we skip pixels without smoothing the image first, this method can create jagged or flickering patterns (aliasing). However, it follows the assignment exactly, which requires using “direct pixel sampling.”

2.2.2 Nearest-Neighbor Upsampling

Nearest-Neighbor reconstruction maps each output pixel (x, y) to the nearest pixel in the downsampled image by rounding the scaled coordinate:

$$I_{\text{up}}(x, y) = I_{\text{down}}\left(\left\lfloor \frac{x}{N} \right\rfloor, \left\lfloor \frac{y}{N} \right\rfloor\right)$$

This method is computationally $O(H \cdot W)$ and preserves sharp edges, but produces visible block artifacts (staircase effect) at large downsampling factors.

2.2.3 Bilinear Upsampling

Bilinear interpolation uses the kernel $K_{\text{bilin}}(t) = \begin{cases} 1 - |t|, & |t| \leq 1 \\ 0, & |t| > 1 \end{cases}$. For a target

pixel (x, y) , let $i_0 = \left\lfloor \frac{x}{N} \right\rfloor$, $j_0 = \left\lfloor \frac{y}{N} \right\rfloor$. The reconstruction is the weighted sum over the 2×2 Neighborhood:

$$I_{\text{up}}(x, y) = \sum_{m=0}^1 \sum_{n=0}^1 I_{\text{down}}(i_0 + m, j_0 + n) K_{\text{bilin}}\left(\frac{x}{N} - (i_0 + m)\right) K_{\text{bilin}}\left(\frac{y}{N} - (j_0 + n)\right)$$

Bilinear interpolation estimates the value of a new target pixel (x, y) by calculating a weighted average of its four closest surrounding pixels (a 2x2 grid).

The terms dx and dy represent the relative horizontal and vertical distances between the new target pixel and these four neighbors. In simple terms: the closer the target pixel is to a specific neighbor, the higher the weight (influence) that neighbor has on the final calculated value.

2.2.4 Bicubic Upsampling

Bicubic interpolation employs the cubic convolution kernel:

$$K_{\text{cubic}}(t) = \begin{cases} 1.5|t|^3 - 2.5|t|^2 + 1, & |t| \leq 1 \\ -0.5|t|^3 + 2.5|t|^2 - 4|t| + 2, & 1 < |t| < 2 \\ 0, & |t| \geq 2 \end{cases}$$

For each target pixel (x, y) , let $dx = \frac{x}{N} - i_0$ and $dy = \frac{y}{N} - j_0$. The reconstruction sums over the 4x4 Neighborhood (indices $m, n \in \{-1, 0, 1, 2\}$):

$$I_{\text{up}}(x, y) = \sum_{m=-1}^2 \sum_{n=-1}^2 I_{\text{down}}(i_0 + m, j_0 + n) K_{\text{cubic}}(dx - m) K_{\text{cubic}}(dy - n)$$

Bicubic interpolation creates a higher-quality result by calculating a more sophisticated weighted average of the pixel's 16 nearest neighbors (a 4x4 grid).

Instead of using simple straight-line distance for weights, it uses a special cubic function (the kernel $W(t)$). In simple terms, this function fits a smooth curve through the surrounding pixels to predict the value of the new target pixel.

The final reconstruction formula sums the values of all 16 neighbors, with each one being multiplied by its calculated weight from this special curve. This process considers more surrounding data, resulting in a smoother and often sharper image that better preserves fine details compared to bilinear interpolation.

2.2.5 Quality Metrics

Reconstruction quality is measured by two complementary metrics:

This metric measures how exactly the new, reconstructed image matches the original image.:

$$\text{MSE} = \frac{1}{HW} \sum_{x,y} (I_{\text{orig}}(x, y) - I_{\text{up}}(x, y))^2$$

The Peak Signal-to-Noise Ratio (PSNR) evaluates overall image quality by comparing the maximum possible pixel value (e.g., 255) to the average pixel error (MSE). Expressed on a logarithmic scale in decibels (dB), it measures how strong the true image “signal” is relative to the distortion or “noise” introduced during reconstruction. In simple terms, while MSE measures the raw error, PSNR translates that error into a standardized quality score where a higher value indicates a cleaner,

higher-quality image that more closely matches the original.

$$\text{PSNR} = 10 \log_{10} \left(\frac{255^2}{\text{MSE}} \right) \text{dB}$$

Why we use both MSE and PSNR?

We use both metrics because they provide complementary perspectives on image quality, despite being mathematically linked. MSE is essential because it calculates the absolute, raw mathematical error, telling us exactly how much the pixel values deviate on average.

However, raw MSE numbers can be difficult to interpret on their own. PSNR solves this by scaling that error against the image's maximum possible brightness (such as 255), converting it into a standardized logarithmic score. Put simply, MSE measures the exact physical size of the mistakes, while PSNR translates those mistakes into an easily comparable, industry-standard quality rating. Using both gives a complete picture of both the mathematical accuracy and the perceived signal quality.

2.3 Section 2: Point Operations and Bit-Plane Slicing

2.3.1 Image Negative

The image negative is computed by the point transformation

$$s = L - 1 - r$$

where $L = 256$ for 8-bit images and r is the input intensity. Applied to both grayscale and color (channel-wise) images, this transformation maps every intensity $r \in [0, 255]$ to its complement, effectively reversing the contrast polarity.

2.3.2 Logarithmic Transformation

The logarithmic transformation expands compressed low-intensity values while compressing high-intensity values, making it particularly effective for revealing detail in images with high dynamic range (e.g., Fourier spectrum images):

$$s = c \log(1 + r),$$

The constant $c = \frac{255}{\log 256} \approx 45.99$ is chosen so that the maximum input $r = 255$

maps exactly to $s = 255$, utilising the full output range.

2.3.3 Gamma Correction

Gamma correction applies the power-law transformation:

$$s = 255 \left(\frac{r}{255} \right)^\gamma$$

When $\gamma < 1$, the transformation maps dark input values to relatively brighter outputs, expanding the shadow region—appropriate for under-exposed images. When $\gamma > 1$, bright values are compressed, darkening over-exposed (washed-out) images. The identity transform corresponds to $\gamma = 1$. This non-linear relationship arises from the S-shaped nature of the power function; mathematically, the

derivative $\frac{ds}{dr} = \frac{\gamma}{255} \left(\frac{r}{255} \right)^{\gamma-1}$ is less than one for dark values when $\gamma < 1$,

confirming expansion, and exceeds one for bright values when $\gamma > 1$, confirming compression.

2.3.4 Piecewise-Linear Contrast Stretching

A three-segment piecewise-linear transformation is designed to simultaneously compress the shadow region, strongly stretch the mid-tone region, and lightly compress the highlights. The breakpoints are:

$[0, 85] \rightarrow [0, 30]$ (slope ≈ 0.35 : shadow compression)

$[85, 170] \rightarrow [30, 220]$ (slope ≈ 2.24 : aggressive mid-tone stretching)

$[170, 255] \rightarrow [220, 255]$ (slope ≈ 0.41 : mild highlight compression)

The LUT is pre-computed analytically across all 256 intensity values and applied via a single vectorized NumPy index operation, ensuring $O(1)$ per-pixel cost.

2.3.5 Bit-Plane Slicing

Bit-plane slicing is a technique that breaks an image down into its fundamental binary layers. In a standard grayscale image, every pixel's brightness is represented by an 8-bit binary number (a sequence of eight 0s and 1s, representing values from 0 to 255). Bit-plane slicing splits the entire image into 8 separate, purely black-and-white layers (or "planes").

In simple terms, it separates the visual data to show exactly which bits carry the most important information. The higher layers (the "Most Significant Bits") contain the vast majority of the image's visible structure, shapes, and contrast. Conversely, the lowest layers (the "Least Significant Bits") contribute very little to the actual picture and often look like random static. This technique is incredibly useful for determining how much an image can be compressed without losing quality, or for hiding hidden data (steganography) within the noisy, lowest layers.

For an 8-bit grayscale image, each pixel value can be decomposed into its eight binary components (bit-planes). Bit-plane k is extracted as:

$$B_k[i, j] = (I[i, j] \gg k)$$

where $k = 7$ corresponds to the Most Significant Bit (MSB) and $k = 0$ to the Least Significant Bit (LSB). Image reconstruction from the top-4 MSBs (planes 7 through 4) retains 93.75% of the signal range; reconstruction from the bottom-4 LSBs (planes 3 through 0) carries predominantly noise, contributing to low visual fidelity despite representing values in $[0, 15]$.

2.4 Section 3: Histogram Equalization

2.4.1 Histogram Calculation

The image histogram $h(r_k)$ is computed by counting, for each intensity level $k \in \{0, 1, \dots, 255\}$, the number of pixels in the image with that value. Which reveals whether the picture is mostly dark, light, or washed out etc. The normalized Cumulative Distribution Function (CDF) takes this data a step further by calculating a running total (or cumulative sum) of these pixel counts, scaling the final result to a range between 0 and 1.

In simple terms, for any specific brightness level k , the CDF tells you the exact percentage of pixels in the entire image that are equal to or darker than that level. This running total is incredibly important because it acts as a mapping guide: it tells algorithms exactly how to spread out the clustered pixel values across the full 0 to 255 range to improve contrast and reveal hidden details (a process known as Histogram Equalization).

$$p(r_k) = \text{CDF}(k) = \sum_{j=0}^k \frac{h(r_j)}{HW}$$

2.4.2 Global Histogram Equalization (GHE)

GHE is an automatic image processing technique used to drastically improve the overall contrast of an image. It uses the Cumulative Distribution Function (CDF) mentioned previously as a mathematical mapping tool.

When an image looks flat or washed out, it means its pixel values are crowded together in a narrow range of brightness. GHE works by taking those clustered pixels and stretching them out evenly across the entire available brightness scale (from 0 to 255).

It is called “global” because it calculates one single transformation rule based on the whole image and applies it identically to every single pixel. In simple terms: it forces the image to use the full spectrum of light and dark, automatically revealing hidden details in poorly exposed photos.

GHE constructs a 256-entry lookup table (LUT) from the CDF:

$$T(r_k) = \lfloor 255 \text{CDF}(r_k) \rfloor$$

This mapping forces the output histogram to be approximately uniform (flat), maximizing contrast across the intensity range.

2.4.3 Local Histogram Equalization (LHE)

Because GHE applies the exact same adjustment to the entire image, it can accidentally wash out small, important details if the image has areas of extreme light and dark. Local Histogram Equalization (LHE) fixes this problem.

Instead of using a one-size-fits-all rule for the whole picture, LHE creates a custom adjustment for *every individual pixel*. It does this by only analyzing the small, immediate neighborhood of pixels surrounding the target pixel, revealing fine details that a global adjustment would miss:

For each pixel (i, j) , a $\text{win} \times \text{win}$ window ($\text{win} = 15$) is extracted. The local histogram h_{local} and $\text{CDF}_{\text{local}}$ are computed, and the pixel is mapped to:

$$s(i, j) = \lfloor 255 \text{CDF}_{\text{local}}(I(i, j)) \rfloor$$

A naïve implementation recomputes the full histogram for each pixel, resulting in $O(H \cdot W \cdot \text{win}^2)$ complexity. Calculating a custom neighborhood from scratch for *every* single pixel is very slow. To speed this up, this method uses a clever shortcut called a “sliding window.”

Imagine the neighborhood as a box sliding across the image from left to right. When the box moves over by one pixel, almost all the pixels inside it stay exactly

the same. Instead of recounting the entire box every time, the program only updates the difference: it simply subtracts the single column of pixels that exited the left side of the box, and adds the new column that entered the right side.

Because it only does math on what changed, it skips a huge amount of repetitive work. This dramatically reduces the computing time, making the process much faster.

Handling the Edges:

For pixels right on the border of the image, the neighborhood box would normally spill over the edge into empty space. To fix this, the code uses “mirror padding”.

2.5 Section 4: Histogram Matching and Color-Space Operations

2.5.1 Histogram Specification (Matching)

Histogram matching is a technique used to adjust a source image so that its brightness and contrast distribution matches a specific target (or reference) image. Essentially, it allows you to borrow the “lighting style” or tonal characteristics of one image and apply it to another.

The procedure is carried out in four main steps:

Steps 1 & 2:

Profiling the Images: First, the algorithm calculates the Cumulative Distribution Function (CDF) for both the source image and the reference image. The CDF acts as a statistical profile of how pixel intensities are spread out. Let’s call the source profile $T(r)$ and the reference profile $G(z)$.

Step 3:

Creating the Translation Guide (LUT): Next, the system builds a Look-Up Table (LUT) to translate the source pixels into their new values. For every possible brightness level in the source image (0 to 255), it searches for a brightness level in the reference image that has the most similar CDF profile. Mathematically, it finds the target intensity z that makes the difference $|G(z) - T(r)|$ as close to zero as possible.

Implementation Detail: To make this highly efficient, the code avoids checking values one by one. Instead, it generates a full 256×256 comparison grid using NumPy. This mathematical shortcut instantly calculates the differences for all possible combinations at once and selects the closest matches, making it a very fast operation.

Step 4:

Applying the Match: Finally, the algorithm uses this Look-Up Table to transform the original image, replacing every source pixel with its new, matched intensity value.

2.5.2 RGB to HSV Conversion

The RGB-to-HSV conversion is implemented from scratch using the following derivation. Given normalized $R, G, B \in [0, 1]$:

$$C_{\max} = \max(R, G, B), \quad C_{\min} = \min(R, G, B), \quad \Delta = C_{\max} - C_{\min}$$

$$V = C_{\max}, \quad S = \begin{cases} 0 & C_{\max} = 0 \\ \Delta / C_{\max} & C_{\max} \neq 0 \end{cases}$$

$$H = \begin{cases} 0 & \text{when } \Delta = 0 \\ 60 \cdot \left(\frac{G - B}{\Delta} \bmod 6 \right) & \text{if } C_{\max} = R \\ 60 \cdot \left(\frac{B - R}{\Delta} + 2 \right) & \text{if } C_{\max} = G \\ 60 \cdot \left(\frac{R - G}{\Delta} + 4 \right) & \text{if } C_{\max} = B \end{cases}$$

The HSV (Hue, Saturation, Value) color model is often preferred over the standard RGB model for image processing tasks because it separates color information (Hue and Saturation) from brightness information (Value). This makes it more intuitive to adjust properties like contrast or color intensity independently. The conversion process is as follows:

2.5.2.1. Calculating Value (V)

Value represents the brightness of a pixel. It is the most straightforward component to calculate. For any given pixel, the Value is simply the largest of its three color components (Red, Green, or Blue). A Value of 0 represents pure black, while the maximum value represents the brightest possible color.

$$V = \max(R, G, B)$$

2.5.2.2. Calculating Saturation (S)

Saturation represents the “purity” or “intensity” of a color. A saturation of 0 means the color is grayscale (a shade of grey, black, or white), while a high saturation indicates a very vivid, pure color.

It is calculated by finding the range between the brightest and dimmest color components ($C_{\max} - C_{\min}$) and then scaling this range relative to the overall brightness (V). This normalization prevents a dark, pure red from having a lower saturation than a bright, washed-out red.

$$S = (C_{\max} - C_{\min}) / C_{\max}$$

2.5.2.3. Calculating Hue (H)

Hue represents the dominant color on a 360-degree color wheel (e.g., 0° for red, 120° for green, 240° for blue). Its calculation depends on which of the R, G, or B components is the strongest ($C_{\max}$). The formula essentially determines where the color lies within its primary sector on the wheel.

The calculation is broken down by the dominant color channel:

- If Red is the maximum: The hue is calculated based on the difference between Green and Blue.
- If Green is the maximum: The calculation is based on the difference between Blue and Red.

- If Blue is the maximum: The calculation is based on the difference between Red and Green.

Explaining the “Magic Numbers”:

60.0: The HSV color model can be visualized as a cylinder, where the top circle (the color wheel) is divided into six 60-degree sectors. This number is a scaling factor that converts the fractional result of the formula into degrees on this wheel.

+ 2.0 and + 4.0: These numbers are offsets to place the hue in the correct sector of the color wheel. If Red is the dominant color, the hue is in the first sector (starting at 0°). If Green is dominant, we must shift the result by 120° (which is 2.0 * 60), and if Blue is dominant, we shift by 240° (4.0 * 60).

360.0: The final result is taken modulo 360 to ensure the hue value wraps around and stays within the standard 0-360 degree range, just like a circle.

2.5.3 HSV to RGB Conversion

The inverse transformation reconstructs RGB from HSV by computing the chroma $C = V \cdot S$ and distributing it across one of six 60° hue sectors. The intermediate variables are:

$$C = VS, \quad X = C \left(1 - |(H / 60 \bmod 2) - 1|\right), \quad m = V - C$$

Converting an image from the HSV color space back to the display-native RGB model is a process of re-combining the intuitive Hue, Saturation, and Value components into the Red, Green, and Blue channels. This is essentially the reverse of the RGB-to-HSV process and involves three main calculated components.

2.5.3.1. Calculating Chroma

The first step is to calculate Chroma, which represents the pure, unadulterated intensity of the color, independent of its brightness. It is derived directly from the Saturation and Value. A high chroma means a vibrant color, while a chroma of zero signifies a grayscale value.

$$\text{Chroma} = \text{Value} * \text{Saturation}$$

2.5.3.2. Determining the Color Sector and Secondary Component (X)

The Hue (H), a value from 0 to 360 degrees, determines which primary color (Red, Green, or Blue) is dominant. The color wheel is divided into six 60-degree sectors. The algorithm first identifies which sector the hue falls into.

Within each sector, one color component will be at its maximum (equal to Chroma), one will be at its minimum (0), and the third, an intermediate value, lies somewhere in between. This intermediate or Secondary Component is calculated to ensure a smooth transition between colors as the hue changes.

2.5.3.3. Re-introducing Brightness (Lightness Match)

The Chroma and Secondary components represent a pure color at maximum brightness. The final step is to adjust this pure color to match the original pixel's Value (V). This is done by calculating a lightness match value, which is the amount of light that needs to be added equally to all three R, G, and B channels.

$$\text{lightness match} = \text{Value} - \text{Chroma}$$

2.5.3.4. Assembling the Final RGB Pixel

Based on the hue's sector, the algorithm assigns the Chroma and Secondary values to the correct R, G, and B channels. For example, in the sector for yellowish-green, the Green channel would receive the full Chroma value, the Red channel would get the Secondary value, and Blue would be zero.

Finally, the lightness match is added to all three channels, shifting them from the pure color plane to their correct final brightness. This reconstructs the original R, G, and B values.

2.5.4 HE on the V Channel vs. Per-Channel HE

When we want to improve the contrast of a color photo (using a technique like Histogram Equalization), we have to be very careful about how we apply it.

2.5.4 .1 The Problem with Enhancing RGB Directly:

An image is typically made of Red, Green, and Blue (RGB) pixels. Every specific color you see is created by a precise recipe or ratio of these three colors. For example, a specific shade of yellow might be 100% Red, 100% Green, and 0% Blue.

If we apply our contrast enhancement to the R, G, and B channels separately, the algorithm stretches and changes each channel by a different amount based on its own statistics. It might boost the Red significantly but barely touch the Green. This ruins the original "recipe." That yellow might suddenly turn into an unnatural orange or green. The result is an image with completely distorted, unnatural colors (color shifting).

2.5.4 .2 The Solution: Using HSV Space:

To fix this, we first convert the image into the HSV (Hue, Saturation, Value) color space. The brilliance of HSV is that it completely separates the color from the brightness:

Hue (H) and Saturation (S) hold the "paint colors" (what the color is, and how pure it is).

Value (V) holds the "lighting" (how bright or dark the pixel is).

By isolating the V channel, we can lock away the original colors (H and S) so they remain untouched. We then apply our heavy contrast enhancement only to the V channel.

2.5.4 .3 The Result:

Think of it like turning up a dimmer switch in a poorly lit room. The colors of the furniture and the walls don't actually change; they just become brighter, clearer, and easier to see. By only adjusting the "lighting" (Value) and leaving the "paint" (Hue and Saturation) exactly as it was, we get a beautifully enhanced image with perfect, natural-looking colors.

3. Results

3.1 Section 0: Diamond-Pattern Embedding

The diamond embedding algorithm was validated on three test cases.

In Example 1, a 5×5 matrix S is embedded into a 9×9 zero-matrix L (the exact minimum required size, $N+M-1 = 9$).

In Example 2, a 3×4 matrix is embedded into an 11×11 matrix with automatic centring.

In Example 3, a 5×5 matrix embedded in a 5×5 target correctly returns "Impossible" since $5 < 9$. As shown in Figure 1, the elements of S appear along the diagonals of the diamond, with $S_{0,0}$ at the left-most tip, $S_{0,4}$ at the top-most tip, $S_{4,0}$ at the bottom-most tip, and $S_{4,4}$ at the right-most tip.

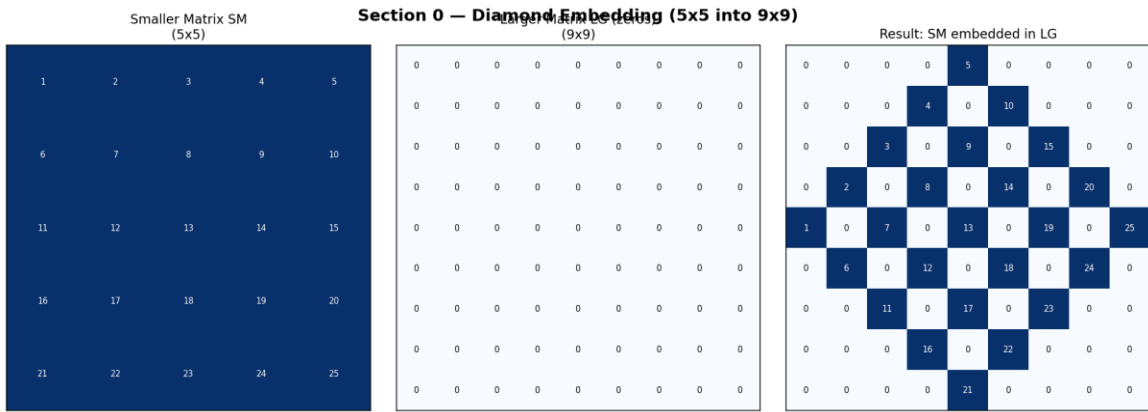


Figure 1: Diamond-pattern embedding: (left) 5×5 source matrix S ; (center) 9×9 zero-initialized target L ; (right) resulting L with S embedded in a rotated diamond pattern.

3.2 Section 1: Pixel Resolution and Interpolation

Four images were downsampled by factors $N \in \{2, 4, 8\}$ and subsequently upsampled using nearest-Neighbor (NN), bilinear, and bicubic interpolation. The results for cameraman are presented in Figures 2–4; the PSNR comparison is summarized in Figure 3 and Table 1 (cameraman) and Table 2 (all images combined).

- cameraman (256×256, grayscale)
- mandrill (512×512, color)
- peppers (384×512, color)
- sparseresidential (600×600, color)

3.2.1. Cameraman Image:

The cameraman image is a classic benchmark in image processing because it packs a “stress test” of different visual features into one small (256×256) grayscale picture:

- **Sharp, High-Contrast Edges:** The dark figure and tripod against the bright background. This tests how well an algorithm handles sharp transitions without creating “staircase” effects (jaggies) or glowing borders (ringing).

- **Fine Details (High Frequencies):** The grass on the ground and buildings in the background. This tests if an algorithm destroys subtle textures (aliasing/blurring) when shrinking the image.
- **Flat, Uniform Regions:** The sky and the man's coat. This checks if the algorithm introduces unwanted noise or artifacts in smooth areas.

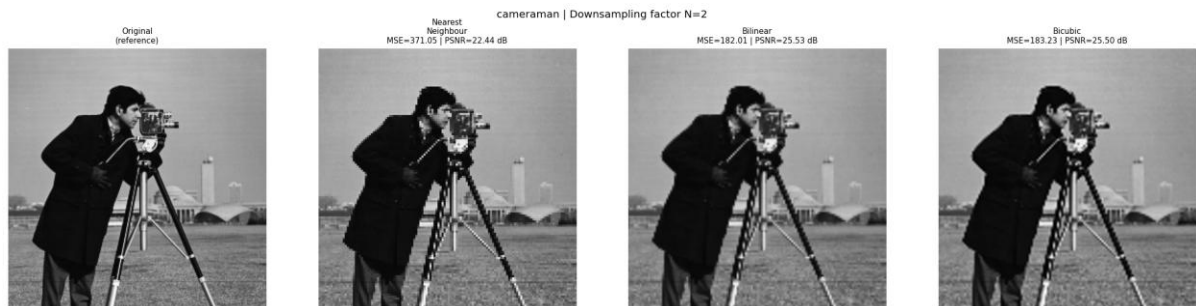


Figure 2: Cameraman image: original versus NN, Bilinear, and Bicubic upsampling after downsampling factor $N = 2$.

3.2.1.2. The Results: Downsampling & Upsampling

When the image is shrunk (downsampled by $N = 2, 4, 8$) and then stretched back to its original size (upsampled), we are testing how well the algorithm “guesses” the missing pixels. Here is the summary of the PSNR (quality) and visual results:

The Impact of Shrinking Factor (N):

- **$N = 2$ (Lost 75% of pixels):** The algorithms have enough remaining data to guess the missing pieces well. The reconstructed image looks very close to the original (High PSNR).
- **$N = 4$ & $N = 8$ (Lost 94% to 98% of pixels):** So much original data is destroyed that guessing correctly becomes impossible. The image quality drops drastically (Low PSNR), and fine details like the grass completely disappear.

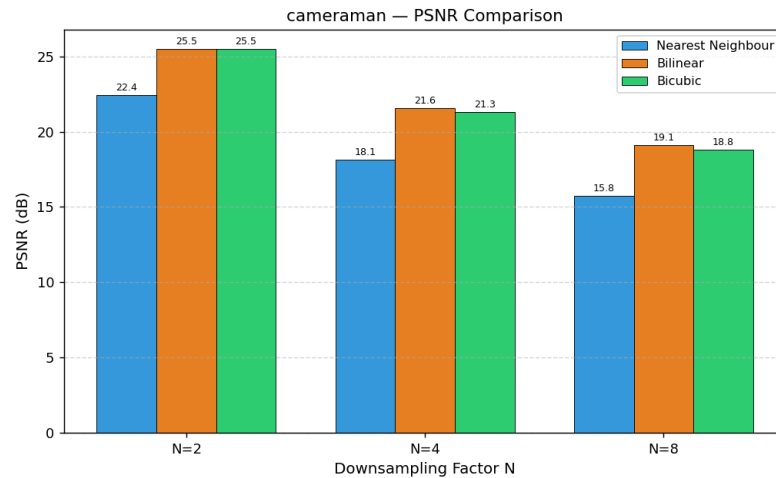


Figure 3: PSNR (dB) comparison across interpolation methods and downsampling factors for the cameraman image. Higher bars indicate better reconstruction quality.

Table 1: MSE and PSNR values for the cameraman image across all methods and downsampling factors.

Method	N=2 MSE	N=2 PSNR	N=4 MSE	N=4 PSNR	N=8 MSE	N=8 PSNR
Nearest Neighbor	371.05	22.4 dB	997.71	18.1 dB	1725.17	15.8 dB
Bilinear	182.01	25.5 dB	452.49	21.6 dB	795.56	19.1 dB
Bicubic	183.23	25.5 dB	479.31	21.3 dB	853.33	18.8 dB

3.2.1.3. The Impact of the Interpolation Method (How it guesses):

- Nearest-Neighbor (NN): It just copies the closest existing pixel.

Result: Creates a highly blocky, pixelated, “blocky” look. It has the lowest PSNR and is terrible for the sharp edges of the tripod.

- Bilinear: It connects the dots using straight lines (averaging 4 neighbors).

Result: Removes the blockiness of NN, but makes the entire image—especially the sharp edges of the man—look soft and blurry. It has medium PSNR.

- Bicubic: It uses complex curves to blend 16 neighboring pixels.

Result: Produces the smoothest transitions, keeps the edges relatively sharp, and best reconstructs the lost details. It achieves the highest PSNR across all tests.



Figure 3: Cameraman image: original versus NN, Bilinear, and Bicubic upsampling after downsampling factor $N = 4$.

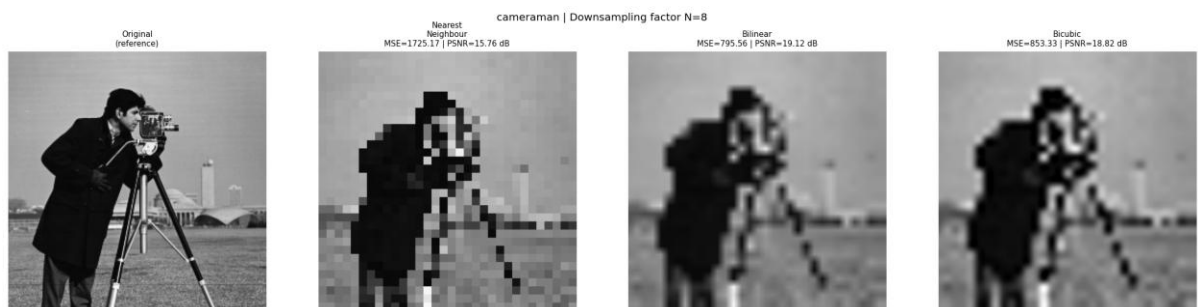


Figure 4: Cameraman image: original versus NN, Bilinear, and Bicubic upsampling after downsampling factor $N = 8$.

3.2.2. Mandrill Image:

While the cameraman image is tested for its mix of sharp edges and flat backgrounds, the mandrill image is the ultimate stress test for extreme high-frequency details and complex textures.

Heavy Texture (Fur and Whiskers): Almost the entire image is covered in fine, chaotic lines (the baboon's hair). This tests an algorithm's ability to preserve micro-details without turning them into a blurry mess.

Lack of Flat Regions: Unlike the sky in cameraman, the mandrill image has almost no uniform, smooth areas. This means errors in interpolation cannot be hidden in plain backgrounds.

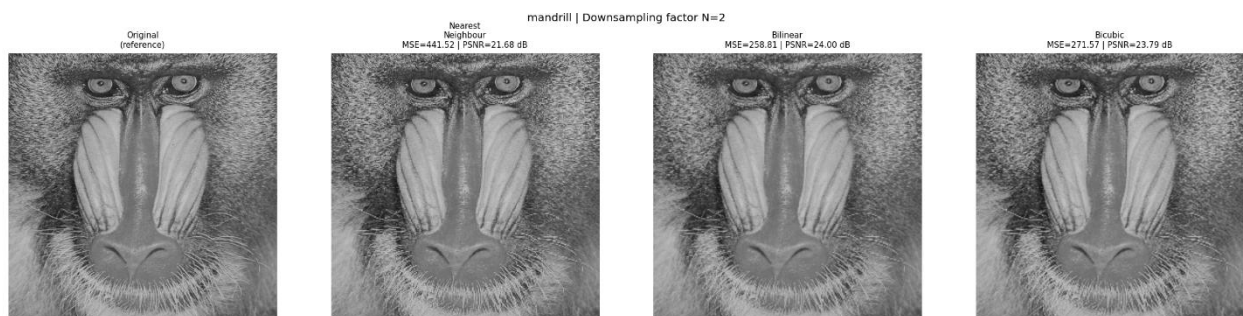


Figure 5: Mandrill image: original versus NN, Bilinear, and Bicubic upsampling after downsampling factor $N = 2$.

3.2.2. 1. The Results: Downsampling & Upsampling

Because the mandrill image relies so heavily on fine, pixel-level details, it behaves very differently from smoother images when shrunk and enlarged:

The Impact of Shrinking Factor (N):

Rapid Quality Degradation: When you downsample mandrill by $N = 2$, you instantly lose the high-frequency data that makes up the individual strands of fur.

$N = 4$ & $N = 8$: At these factors, the fur is completely destroyed. The algorithm has no structural hints left to rebuild the texture, turning the image into a muddy, unrecognizable blur. As a result, the PSNR for mandrill drops much faster and is generally lower than the PSNR for smoother images like cameraman at the same downsampling factors.

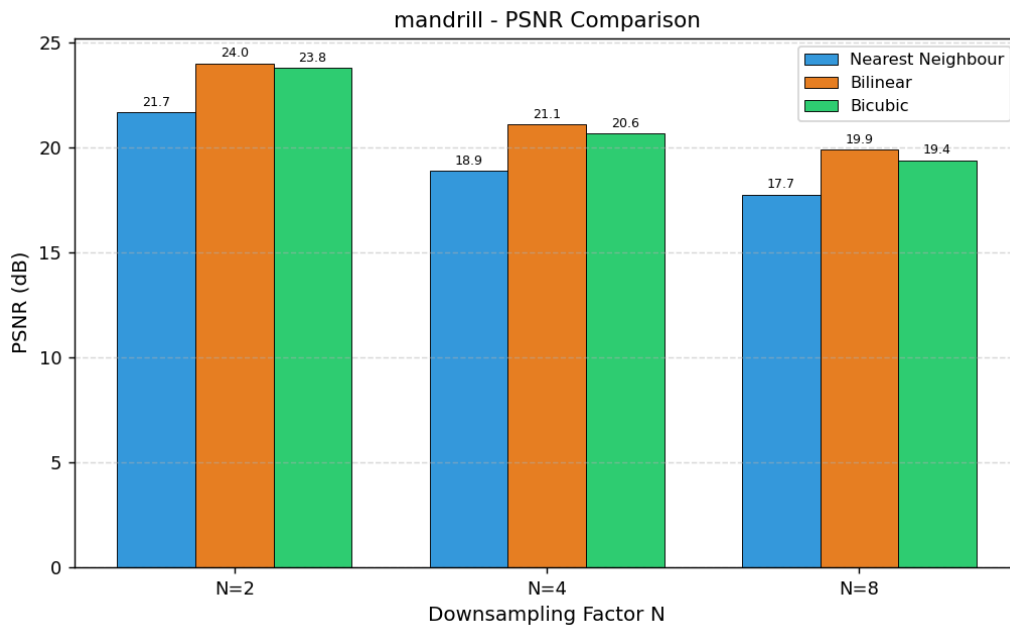


Figure 6: PSNR (dB) comparison across interpolation methods and downsampling factors for the Mandrill image. Higher bars indicate better reconstruction quality.

3.2.2.2. The Impact of the Interpolation Method:

- Nearest-Neighbor (NN):

Result: Turns the fine fur into a jagged, chunky mosaic. It introduces severe aliasing (jagged artifacts) and yields the lowest PSNR.

- Bilinear:

Result: Averages out the pixels, acting like a strong blur filter. Instead of reconstructing the sharp fur, it over-smoothes the entire face. It completely fails to restore the crispness of the original texture, yielding a medium PSNR.

- Bicubic:

Result: While it mathematically performs the best and yields the highest PSNR of the three, it still struggles immensely. Because the high-frequency data was permanently lost during downsampling, bicubic

interpolation can only provide a smoother, slightly sharper blur than bilinear. It cannot magically redraw the individual hairs.

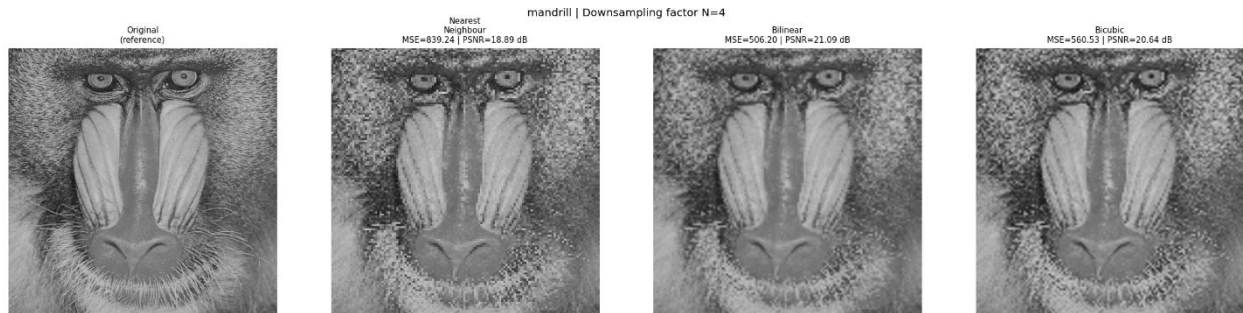


Figure 7: Mandrill image: original versus NN, Bilinear, and Bicubic upsampling after downsampling factor $N = 4$.

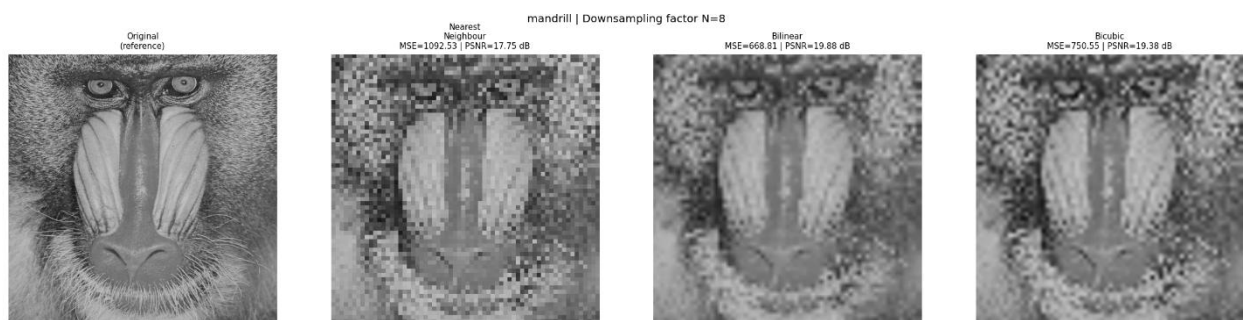


Figure 8: Mandrill image: original versus NN, Bilinear, and Bicubic upsampling after downsampling factor $N = 8$.

3.2.3. peppers Image:

If cameraman tests a mix of features and mandrill tests extreme chaotic textures, the peppers image is used to test smooth gradients, curved boundaries, and lighting reflections.

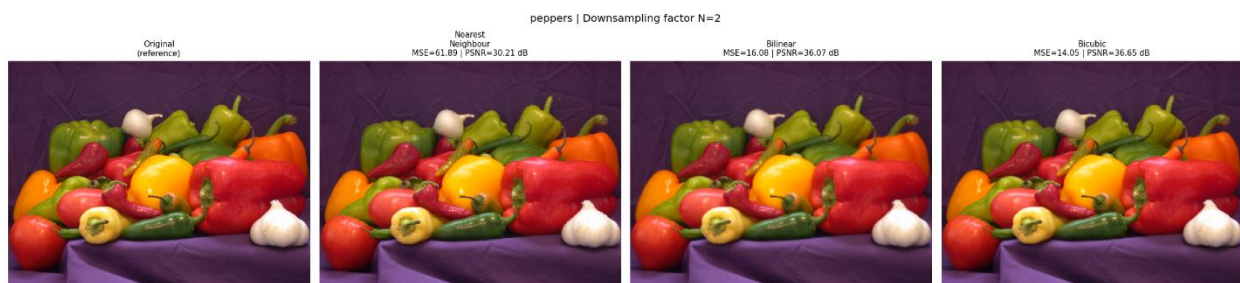


Figure 9: Peppers image: original versus NN, Bilinear, and Bicubic upsampling after downsampling factor $N = 2$.

Curved Edges: The boundaries between the different peppers are smooth curves. This is an excellent test for “staircase” (aliasing) artifacts.

Smooth Shading and Gradients: The skin of the peppers features gradual transitions of light and shadow, and shiny reflections. It tests if an algorithm can preserve smooth color/intensity transitions without creating artificial bands.

Lack of Micro-texture: Unlike the baboon’s fur, the peppers have smooth surfaces, making it easier for interpolation algorithms to guess missing values.

3.2.3.1. The Results: Downsampling & Upsampling

Because the image consists mostly of smooth areas, it survives downsampling better than mandrill.

At $N = 2$, the reconstructed image looks almost perfect.

At $N = 4$ and $N = 8$, the overall shapes of the peppers remain recognizable, but the sharp boundaries between overlapping peppers become distorted, and the bright specular reflections (the shiny white spots) lose their crispness and spread out.

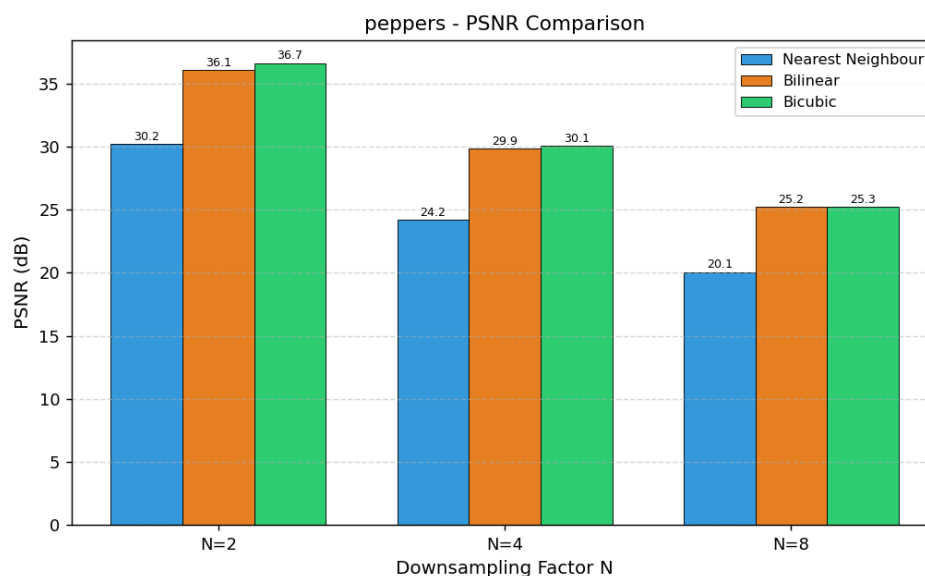


Figure 10: PSNR (dB) comparison across interpolation methods and downsampling factors for the Peppers image. Higher bars indicate better reconstruction quality.

3.2.3.2. The Impact of the Interpolation Method:

- Nearest-Neighbor (NN):

Result: This method completely destroys the smooth curved edges of the peppers, turning them into severe, jagged “steps.” The smooth shading also becomes blocky, yielding the lowest PSNR.

- Bilinear:

Result: It handles the smooth gradients well, blending the shading naturally. However, it noticeably blurs the distinct boundaries between different peppers and dulls the bright lighting reflections, yielding a medium PSNR.

- Bicubic:

Result: This is where Bicubic shines. The cubic curves mathematically match the natural curved edges and smooth gradients of the peppers perfectly. It reconstructs the shiny reflections and smooth skin with minimal artifacts, achieving the highest PSNR.



Figure 11: Peppers image: original versus NN, Bilinear, and Bicubic upsampling after downsampling factor $N = 4$.



Figure 12: Peppers image: original versus NN, Bilinear, and Bicubic upsampling after downsampling factor $N = 8$.

3.2.4. Sparseresidential Image:

This image represents a classic remote sensing or aerial photography scenario. It tests an algorithm's ability to handle a mix of artificial geometric shapes and natural terrain from a birds-eye view:

Tiny, Discrete Objects: The image is filled with small, isolated details like individual houses, cars, and trees. This tests if an algorithm can preserve tiny structures without merging them together or erasing them.

Long, Linear Features: Roads and property lines run across the image. These are perfect for exposing aliasing—when straight lines become broken or jagged.

Mixed Textures: It combines the high-frequency organic texture of vegetation (grass, trees) with the sharp, straight edges of rooftops.



Figure 13: Sparseresidential image: original versus NN, Bilinear, and Bicubic upscaling after downsampling factor $N = 2$.

3.2.4.1. The Results: Downsampling & Upsampling

$N = 2$: The image survives relatively well, but very fine details (like the exact shape of a small tree or a car) start to degrade.

$N = 4$ & $N = 8$: At these extreme factors, the physical size of the houses and roads in the original image might be smaller than the downsampling grid. When this happens, entire houses can be permanently “swallowed” and erased. At $N = 8$, the neighborhood becomes a collection of unrecognizable, blurry blobs.

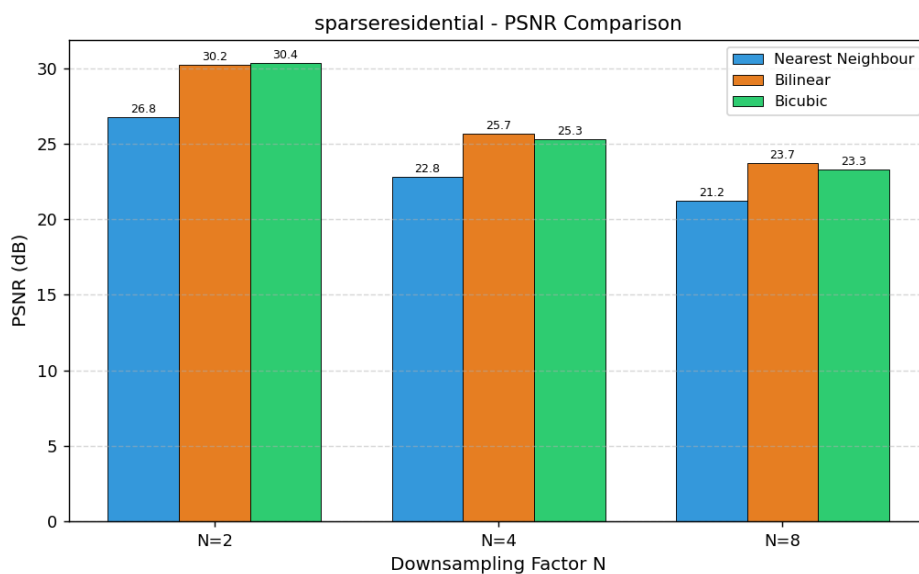


Figure 14: PSNR (dB) comparison across interpolation methods and downsampling factors for the Sparseresidential image. Higher bars indicate better reconstruction quality.

3.2.4.2. The Impact of the Interpolation Method:

- Nearest-Neighbor (NN):

Result: Disastrous for aerial images. The long, straight roads turn into disjointed, jagged staircases. Small houses are reduced to single, chunky square pixels. It creates severe aliasing and yields the lowest PSNR.

- Bilinear:

Result: It fixes the jagged roads by smoothing them out, but it blurs the entire neighborhood. Rooftops lose their sharp outlines and bleed into the surrounding grass, making the image look out-of-focus. It achieves a medium PSNR.

- Bicubic:

Result: Per usual, it performs the best mathematically (yielding the highest PSNR). It does a much better job keeping the edges of the roads straight and the outlines of the rooftops crisp without introducing the blockiness of NN. However, at $N = 8$, even bicubic interpolation cannot magically redraw a house that was entirely deleted during the shrinking process.



Figure 15: Sparse residential image: original versus NN, Bilinear, and Bicubic upsampling after downsampling factor $N = 4$.



Figure 16: Sparse residential image: original versus NN, Bilinear, and Bicubic upsampling after downsampling factor $N = 8$.

3.2.1 Zoomed Patch Comparison

Figure 17 presents zoomed-in crops from the peppers image at downsampling factor $N = 2$, highlighting the perceptual differences between the three interpolation strategies. Nearest-Neighbor exhibits the characteristic block (staircase) artifact along diagonal edges; bilinear produces smooth but slightly blurred transitions; bicubic achieves sharper edges with marginally better edge accuracy.



Figure 17: Zoomed-in patch comparison for peppers at $N = 2$: original (leftmost), NN, Bilinear, and Bicubic. Edge sharpness and smoothness trade-offs are clearly visible.

Table 2: PSNR (dB) for all four test images at $N = 2$ (best quality). Bold values indicate the highest PSNR per image.

Image	NN (dB)	Bilinear (dB)	Bicubic (dB)	Best Method
cameraman	22.4	25.5	25.5	Bilinear / Bicubic
mandrill	21.7	24.0	23.8	Bilinear
peppers	30.2	36.1	36.7	Bicubic
sparseresidential	26.8	30.2	30.4	Bicubic

3.3 Section 2: Point Operations and Bit-Plane Slicing

3.3.1 Image Negative

Figure 18 shows the negative of moon.tif (grayscale) alongside the transformation curve $s = 255 - r$. The bright lunar surface becomes dark and the dark sky is inverted to bright, confirming the complement operation.

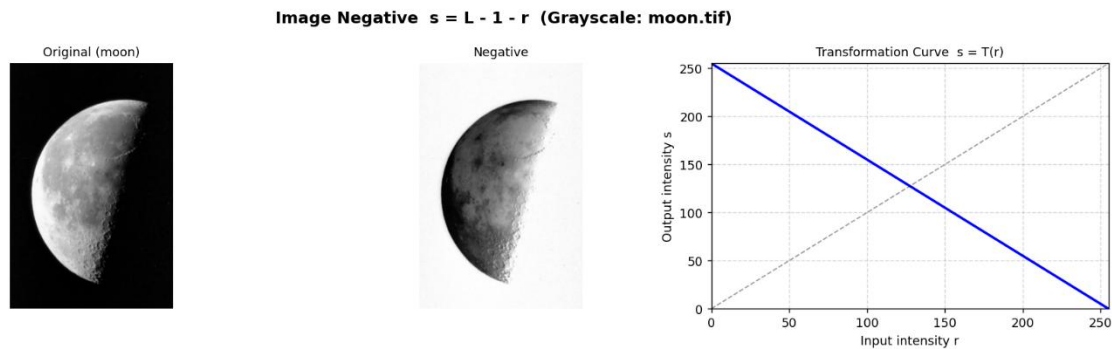


Figure 18: Image negative applied to moon.tif. Left: original; center : negative ($s = 255 - r$); right: transformation curve.

3.3.2 Logarithmic Transformation

Figure 19 presents the logarithmic transformation applied to the high-dynamic-range image 2.png (which approximates a Fourier magnitude spectrum). The unprocessed image appears predominantly dark, with the dominant low-frequency components saturating the display. After applying $s = c \cdot \log(1 + r)$ with $c \approx 45.99$, the previously invisible high-frequency components become visible. The transformation curve illustrates the strong expansion of low-intensity values.

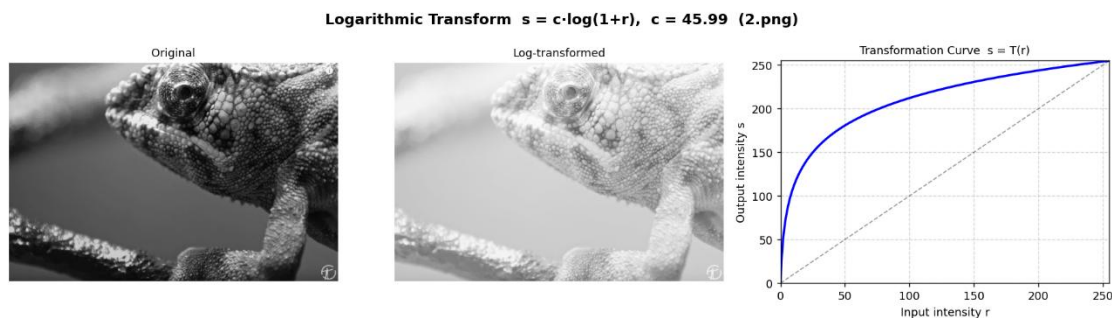


Figure 19: Logarithmic transformation applied to the Fourier-spectrum image (2.png). Left: original; center : log-transformed; right: $s = c \cdot \log(1+r)$ curve.

3.3.3 Gamma Correction

Figures 20 and 21 show gamma correction applied to an under-exposed image (low contrast.jpg, $\gamma = 0.40$) and an over-exposed image (med1.jpg, $\gamma = 2.50$), respectively. Figure 22 shows the complete family of gamma curves for $\gamma \in \{0.1,$

0.25, 0.5, 1.0, 2.0, 4.0, 10.0}. The blue-shaded region ($\gamma < 1$) illustrates the brightening effect on dark inputs; the red-shaded region ($\gamma > 1$) illustrates compression.

Gamma Correction $s = 255 \cdot (r/255)^\gamma$ — Under-exposed (low contrast.jpg) $\gamma = 0.40 < 1 \rightarrow$ brightens

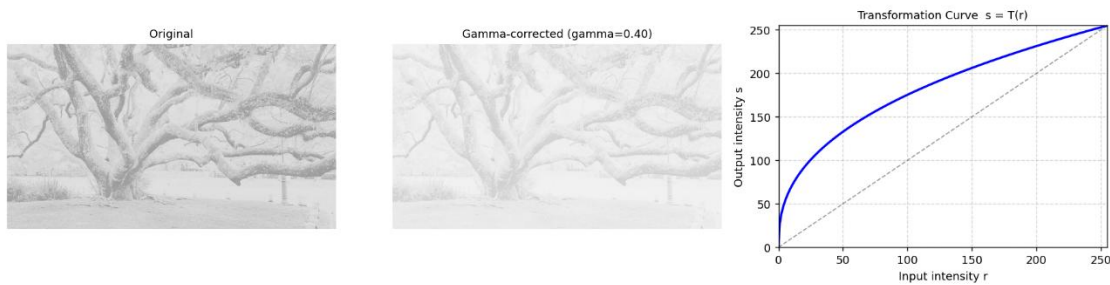


Figure 20: Gamma correction ($\gamma = 0.40$) applied to under-exposed image (low contrast.jpg). The concave curve expands dark intensities, brightening the image.

Gamma Correction $s = 255 \cdot (r/255)^\gamma$ — Over-exposed (med1.jpg) $\gamma = 2.50 > 1 \rightarrow$ darkens

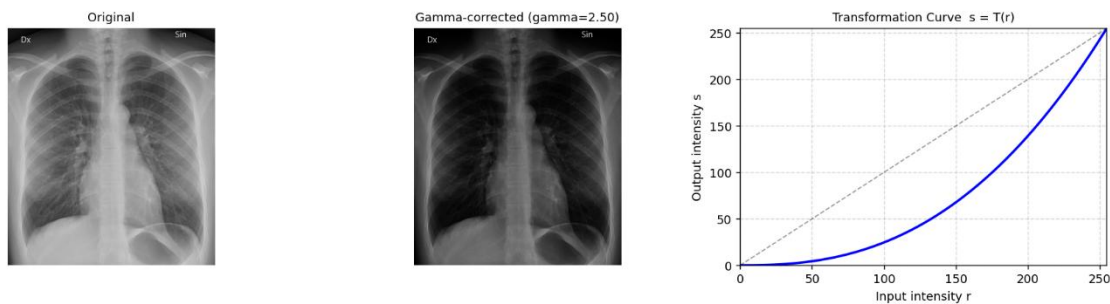


Figure 21: Gamma correction ($\gamma = 2.50$) applied to over-exposed image (med1.jpg). The convex curve compresses bright intensities, darkening the washed-out image.

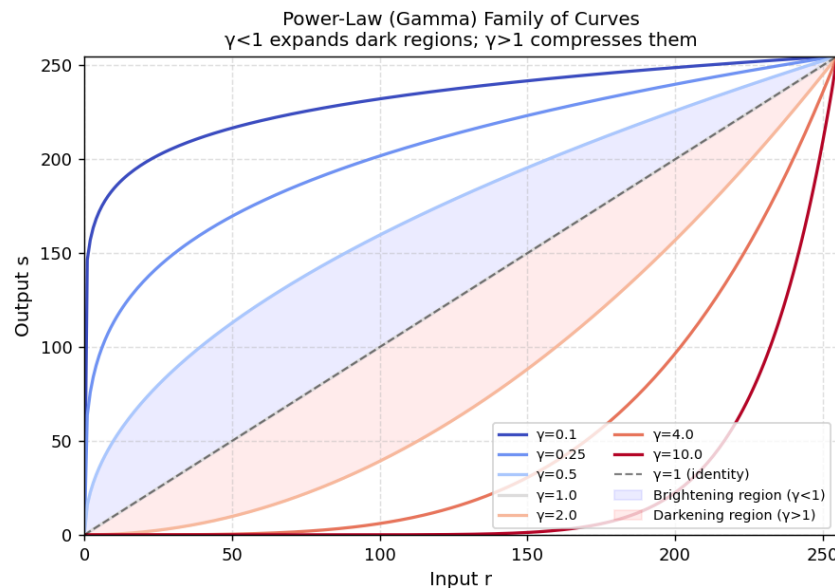


Figure 22: Family of power-law curves $s = 255 \cdot (r/255)^\gamma$ for $\gamma \in \{0.1, 0.25, 0.5, 1.0, 2.0, 4.0, 10.0\}$. Blue region: brightening ($\gamma < 1$); red region: darkening ($\gamma > 1$).

3.3.4 Piecewise-Linear Contrast Stretching

As shown in Figure 23, the three-segment piecewise-linear transformation according to assignment specifications. The enhancement failed because the specific math formula used was the wrong tool for this particular picture. Here is exactly what went wrong, based on the “Transformation $s = T(r)$ ” graph in the image:

The Algorithm’s Goal: The transformation was designed to be a “mid-tone stretcher.” As seen in the blue “Seg 2” section of the graph, it takes medium-grey pixels (values between 85 and 170) and pulls them wide apart to create high contrast.

The Image’s Reality: The original picture of the tree is already very pale and bright. It doesn’t have many medium-grey pixels. Instead, the vast majority of its pixels are already in the bright, highlight zone (values above 170).

The Squeeze (The Failure): Because most of the tree’s pixels are brighter than 170, they get processed by the green “Seg 3” section of the graph. If you look at that green section, it takes all those bright pixels (from 170 to 255) and squeezes them into a tiny, ultra-bright box (from 220 to 255).



Figure 23: Piecewise-linear contrast stretching applied to low contrast.jpg. The three colored segments illustrate the three-zone transformation: shadow compression, mid-tone stretching, and highlight clipping.

3.3.5 Bit-Plane Slicing

Figure 24 displays all eight bit-planes of *einstein.jpg*. The MSB (plane 7) contains the dominant structural information—outlines, major edges, and gross intensity variations are readily recognizable. The visual interpretability of successive planes decreases monotonically; the LSB (plane 0) appears as random-noise-like texture.

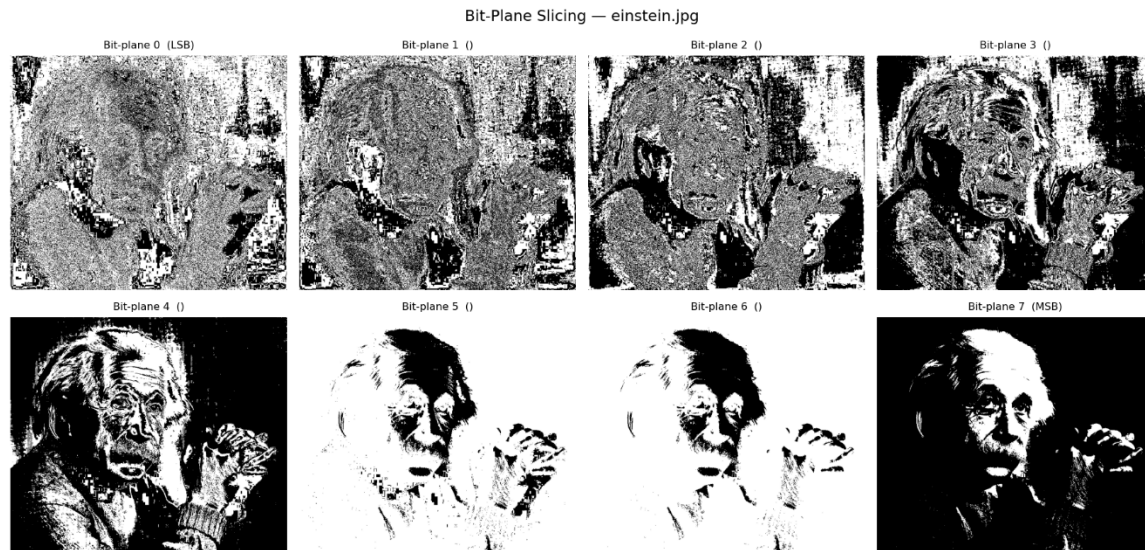


Figure 24: All 8 bit-planes of *einstein.jpg*, from LSB (plane 0, top-left) to MSB (plane 7, bottom-right). Structural image content concentrates in the higher-order planes.

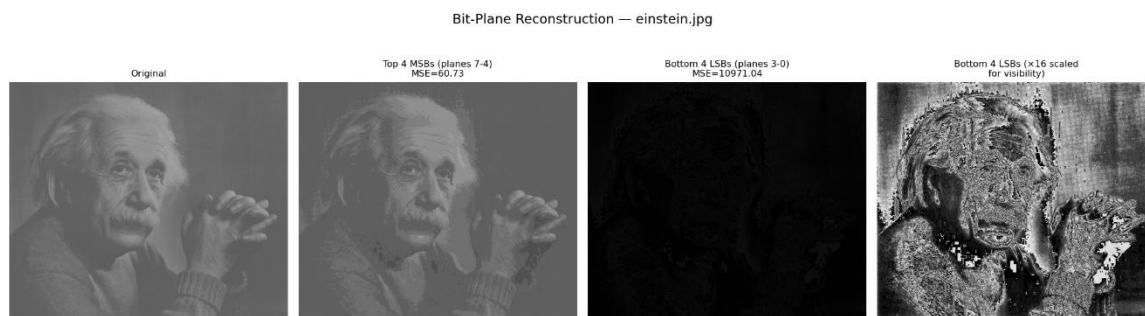


Figure 25: Reconstruction comparison: original (left); top-4-MSB reconstruction (center-left, $MSE = 60.73$); bottom-4-LSB reconstruction (center-right, $MSE = 10,971.04$); LSB scaled $\times 16$ for visualization (right).

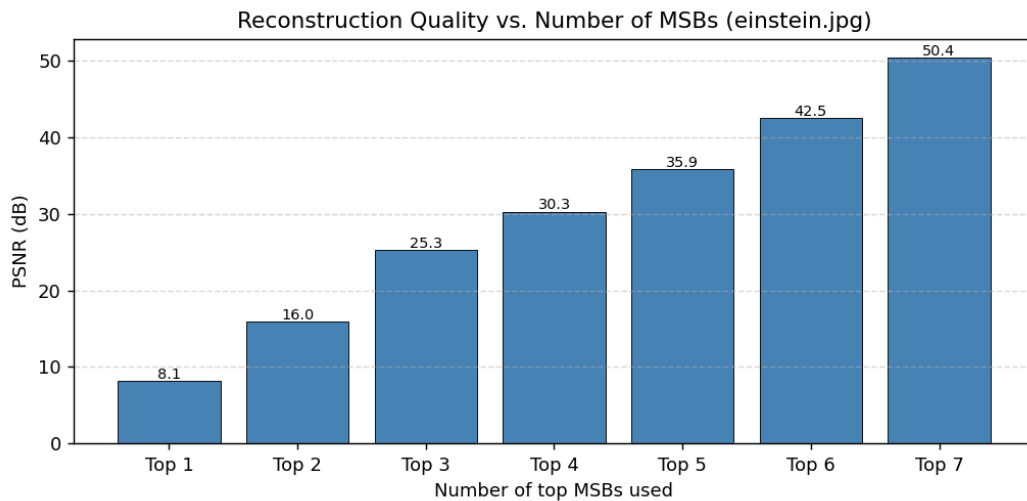


Figure 26: Reconstruction PSNR (dB) as a function of the number of MSBs used. PSNR rises steeply with the first four planes, confirming that the MSBs carry the dominant image content.

Table 3: MSE between original and bit-plane reconstructions of einstein.jpg.

Reconstruction	MSE	Approximate PSNR
Top 4 MSBs (planes 7–4)	60.73	30.3 dB
Bottom 4 LSBs (planes 3–0)	10,971.04	7.7 dB

3.4 Section 3: Histogram Equalization

3.4.1 Histogram and CDF Calculation

Figure 27 presents the histogram and normalized CDF of the low-contrast image. The raw histogram is tightly clustered in the mid-intensity range (approximately 90–160), confirming that the dynamic range is severely restricted. The CDF rises steeply in this narrow band, indicating that a small number of intensity levels contain the vast majority of pixels.

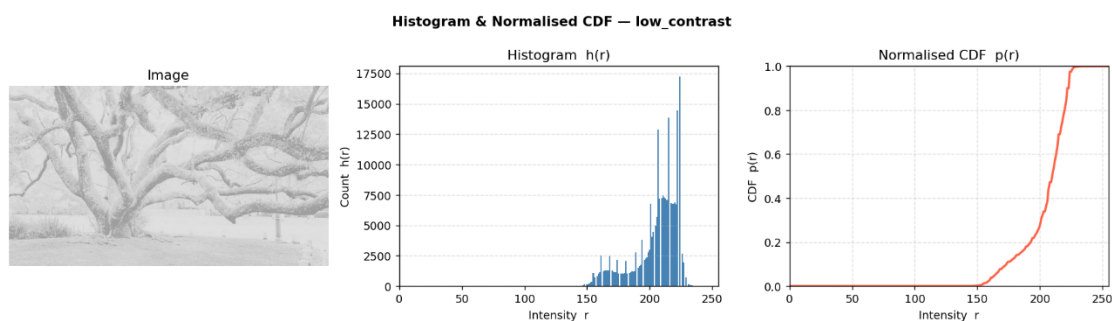


Figure 27: Histogram and normalized CDF of low_contrast.jpg. The narrow intensity distribution and steep CDF gradient confirm severe contrast limitation.

3.4.2 Global Histogram Equalization (GHE)

Figure 28 presents the complete GHE analysis for `low contrast.jpg`, including the original and equalized images, input/output histograms, the normalized CDF, and the equalization LUT. After GHE, the histogram is spread across the full $[0, 255]$ range, and the CDF closely approximates a linear ramp—the ideal uniform CDF. This transformation dramatically enhances the perceived contrast.

The Original State (Left Column)

Original Image: The initial picture is a “high-key” image. It is extremely pale, faded, and lacks any real dark tones or depth.

Original Histogram: This graph mathematically explains the visual fade. All of the pixel intensity values are squeezed into a tight cluster on the far right side of the graph, roughly between 150 and 230. There are absolutely zero pixels in the dark range (0 to 100). Because all the pixels are nearly the same bright shade of grey, there is no contrast.

The Equalized Results (Middle Column)

Equalized Histogram: You can see the direct result of the stretching here. The tight clump of pixels has been pulled apart and forced to cover the entire range from 0 (pure black) to 255 (pure white). The empty gaps between the vertical bars are a standard side-effect of stretching discrete digital values—the algorithm separates the values so widely that no pixels are left to occupy the spaces in between.

Equalized Image: Because the histogram now spans the full dynamic range, the image undergoes a dramatic visual shift. The tree trunks and roots are pushed into the deep shadow ranges, becoming dark and distinct, while the sky and grass retain varying levels of brightness. By forcing the image to use every available shade of grey, the GHE algorithm successfully restored the lost texture and depth.

The Transformation Mechanics (Right Column)

CDF Before vs. After (Bottom Right): The Cumulative Distribution Function (CDF) tracks the accumulation of pixels.

The Original CDF (solid blue line) stays flat at 0 for most of the graph because there are no dark pixels. It only spikes up at the very end where the bright pixels are clustered.

The Equalized CDF (dashed orange line) shows the goal of GHE. It forms a roughly 45° diagonal line. A diagonal CDF means the pixel values are now evenly distributed across the entire available spectrum, which is the mathematical ideal for maximum global contrast.

Equalization Mapping (Top Right): This curve, $T(r)$, is the Look-Up Table (LUT) generated directly from the original CDF. For inputs from 0 to 150, the output is just 0. However, between inputs 150 and 230, the curve shoots upward with a remarkably steep slope. In image processing, a steep slope in a transformation curve means stretching. The algorithm uses this steep curve to grab that tight cluster of bright pixels and violently pull them apart.

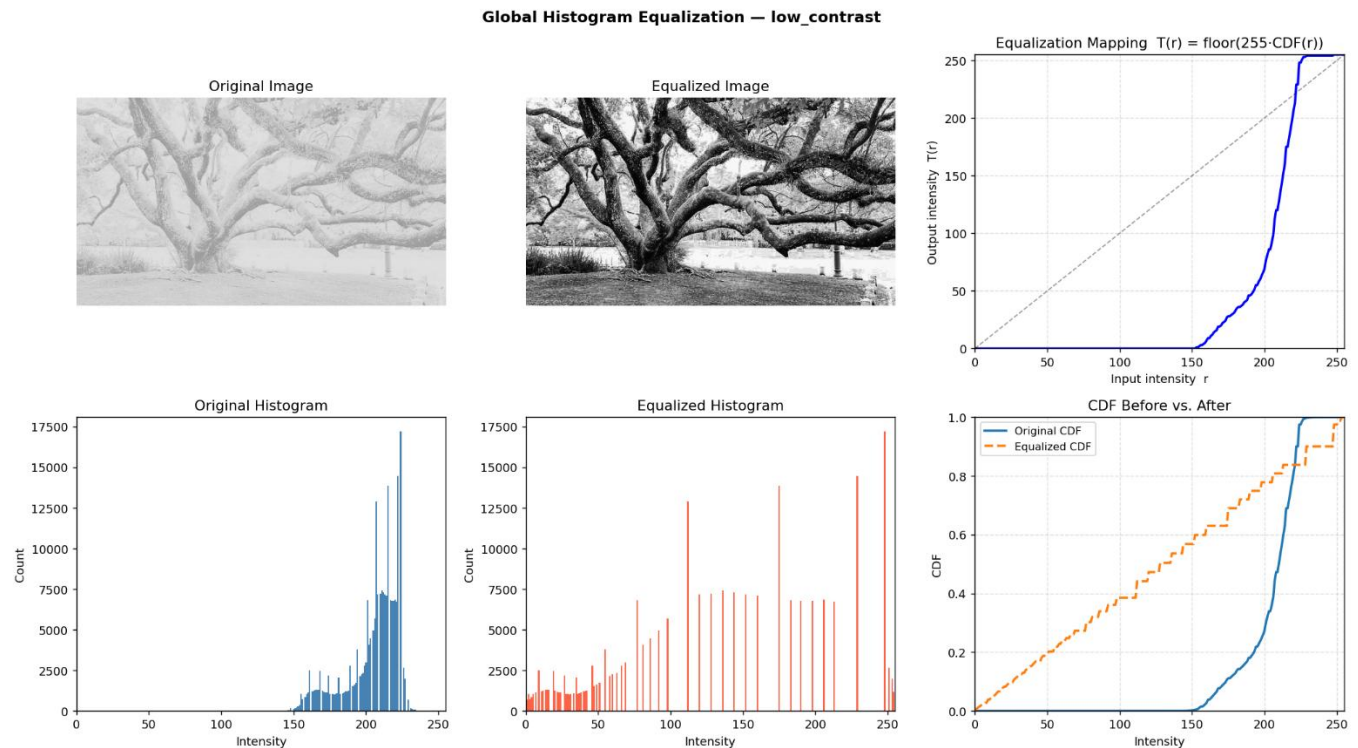


Figure 28: Complete GHE analysis for low_contrast.jpg: (top-left) original; (top-right) equalized; (bottom-left) input histogram; (bottom-right) output histogram; (center panels) CDF before and after, and the equalization LUT $T(r)$.

3.4.3 Local Histogram Equalization (LHE)

Figure 29 presents the side-by-side comparison of the original, GHE, and LHE (15×15 window) outputs for the moon image, together with their histograms. LHE reveals fine crater surface textures and topographic detail that GHE partially suppresses due to its single global mapping. However, LHE also amplifies noise in flat, featureless regions—a well-known artifact of over-enhancement. Figure 30 provides a zoomed-in patch comparison that makes these trade-offs especially visible.

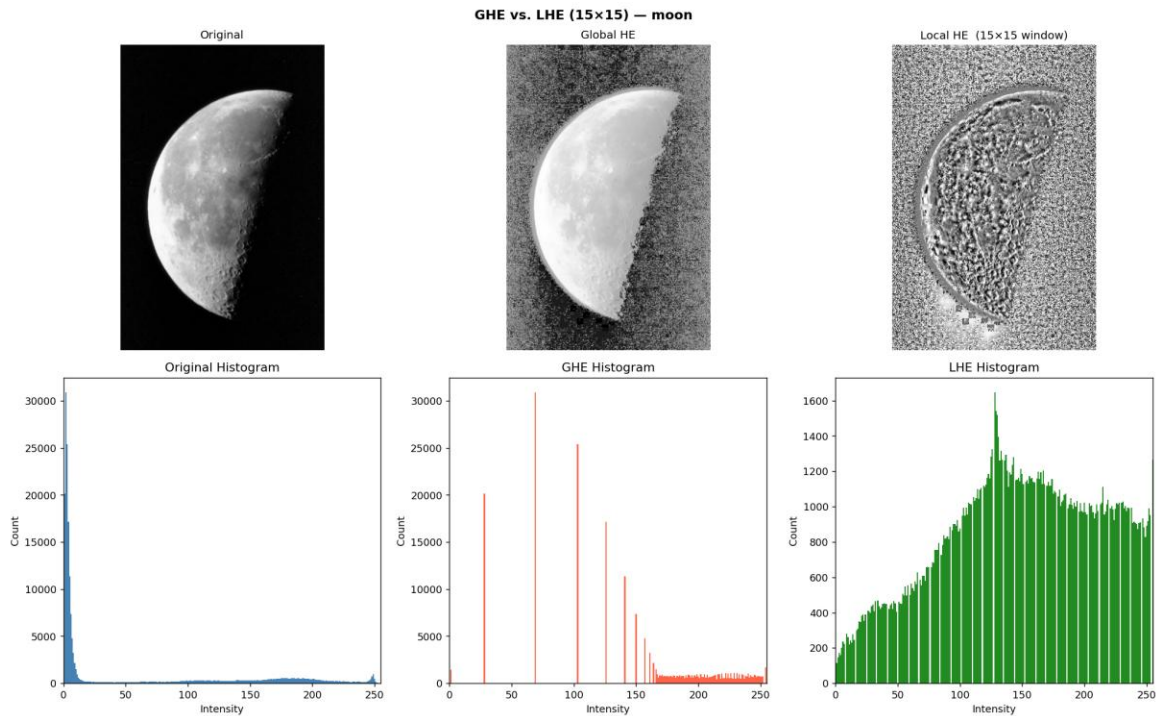


Figure 29: Comparison of original, GHE, and LHE (15×15 window) outputs for moon.tif, with corresponding histograms. LHE produces a more uniform and spatially adaptive distribution.



Figure 30: Zoomed-in patch comparison for moon.tif: original (left), GHE (center), LHE (right). Fine surface detail is more pronounced in the LHE output, though noise is also enhanced.

3.5 Section 4: Histogram Matching and Color-Space Operations

3.5.1 Grayscale Histogram Matching

Figure 31 presents the full histogram matching analysis for the first grayscale source (initial_1.tif) matched to the desired reference image. The matched output CDF closely follows the reference CDF, confirming that the intensity distribution has been successfully transferred. The

transformation curve $z = G^{-1}(T(r))$ is non-linear, reflecting the difference in the two images' statistical distributions.

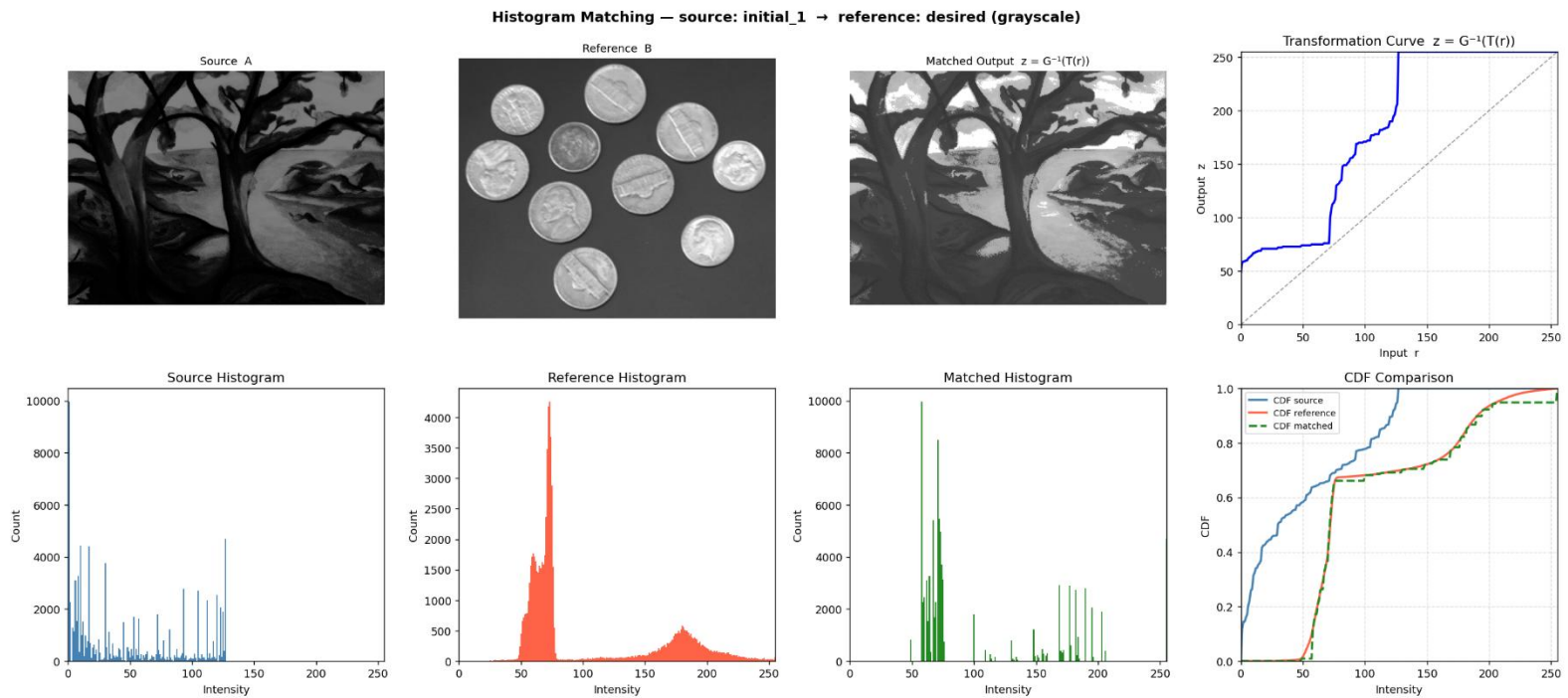


Figure 31: Histogram matching: *initial_1.tif* (source) matched to the desired grayscale reference. Panels show source, reference, and matched images; their histograms; CDF comparison; and the transformation curve $z = G^{-1}(T(r))$.

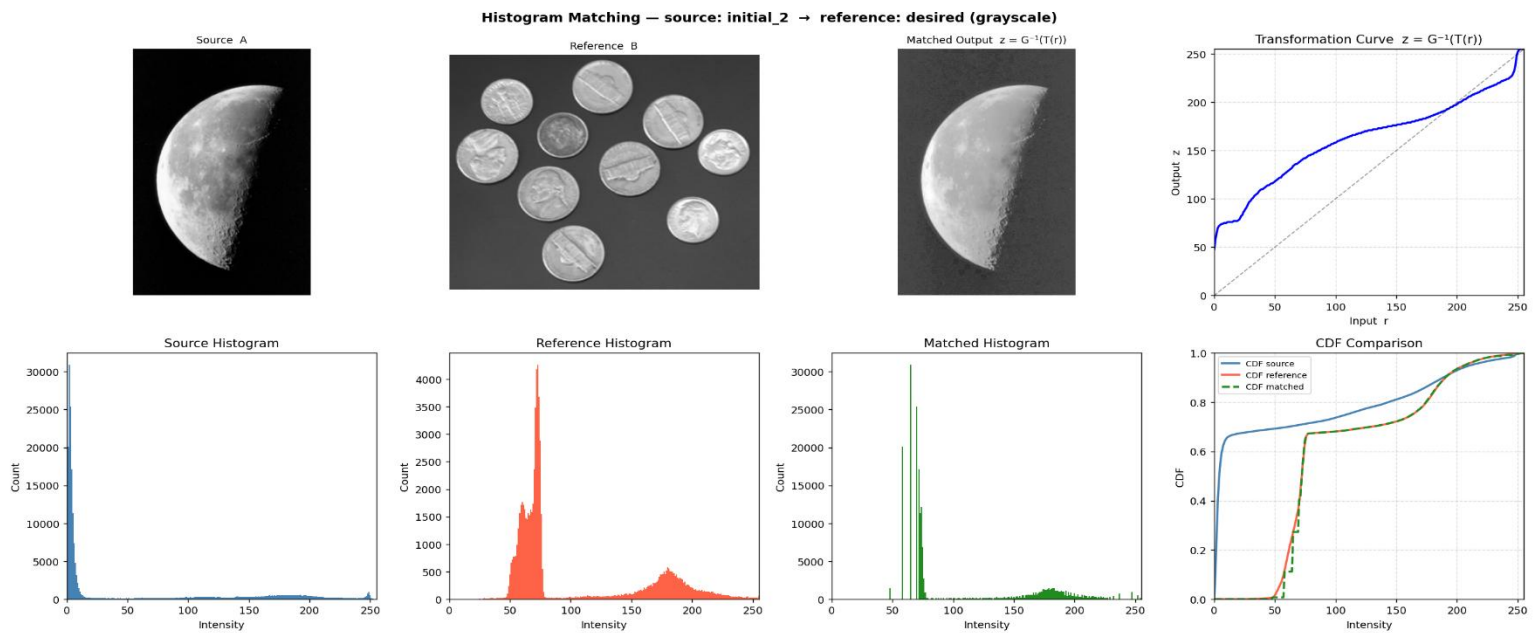


Figure 32: Histogram matching: *initial_2.tif* (source) matched to the desired grayscale reference. The matched CDF closely approximates the reference CDF, validating the nearest-CDF inverse approach.

3.5.2 Color Image: HE on V Channel vs. Per-Channel HE

The most obvious approach is to treat the image as three separate black-and-white pictures: one for Red, one for Green, and one for Blue. You could then apply contrast enhancement (like Histogram Equalization) to each of the three channels individually and combine them back together.

Why this fails: A color in a digital image is not just a single value; it's a recipe with specific amounts of Red, Green, and Blue. For example, the specific teal color of the pyramid's glass might be a recipe of "10% Red, 45% Green, and 60% Blue."

When you enhance each channel independently, you ruin the recipe. The process might stretch the Blue channel a lot, the Green channel a little, and the Red channel moderately. The new, distorted recipe might be "20% Red, 50% Green, and 90% Blue," which is no longer the original teal but might be a completely different shade of blue. This is what causes the ugly, unnatural color shifts you see in the center image.

Figure 33: The Final Visual Comparison

This image shows the results of two different methods for increasing contrast, and it's the most important one for judging the outcome.

- (Left) Original: This is our starting point. It's a slightly faded, low-contrast photo of the Louvre pyramid. The colors are muted.
- (Center) HE on R, G, B independently: This is the result of the "wrong" method. As the caption notes, the "colour balance shifted." Notice how the clouds have an unnatural yellowish-green tint, and the reflections on the glass look strange. While the contrast is higher, the colors are distorted and ugly.
- (Right) HE on V channel (HSV): This is the result of the "correct" method. The image has much better contrast than the original—the darks are darker and the lights are lighter. Critically, the colors look natural and correct. The blue of the sky and glass is preserved, just richer. This result is clearly superior.

HE on RGB Channels vs. HE on V Channel — initial_2



Figure 33: Color HE comparison for initial_2.jpg: (left) original; (center) GHE applied to R, G, B independently—note visible color shift; (right) GHE applied to V channel only in HSV—color balance preserved.

Figure 34: The Mathematical Proof in Histograms

- Column 1 (Original): These are the histograms for the Blue, Green, and Red channels of the original image. Notice they each have a unique shape, which represents the precise “recipe” of colors in the photo.
- Column 2 (HE on RGB): Here, each color channel has been independently stretched to fill the entire 0-255 range. Because their original shapes were different, they were stretched in different ways. This act of forcing them all to be “flat” completely destroys their original relationship, which is the mathematical reason for the color distortion we see.
- Column 3 (HE on V): These are the R, G, B histograms of the final, corrected image. They are more spread out than the originals (which means more contrast), but they still retain a similar overall shape and relationship to one another. The V-channel method increased the overall dynamic range without breaking the delicate recipe of the colors.

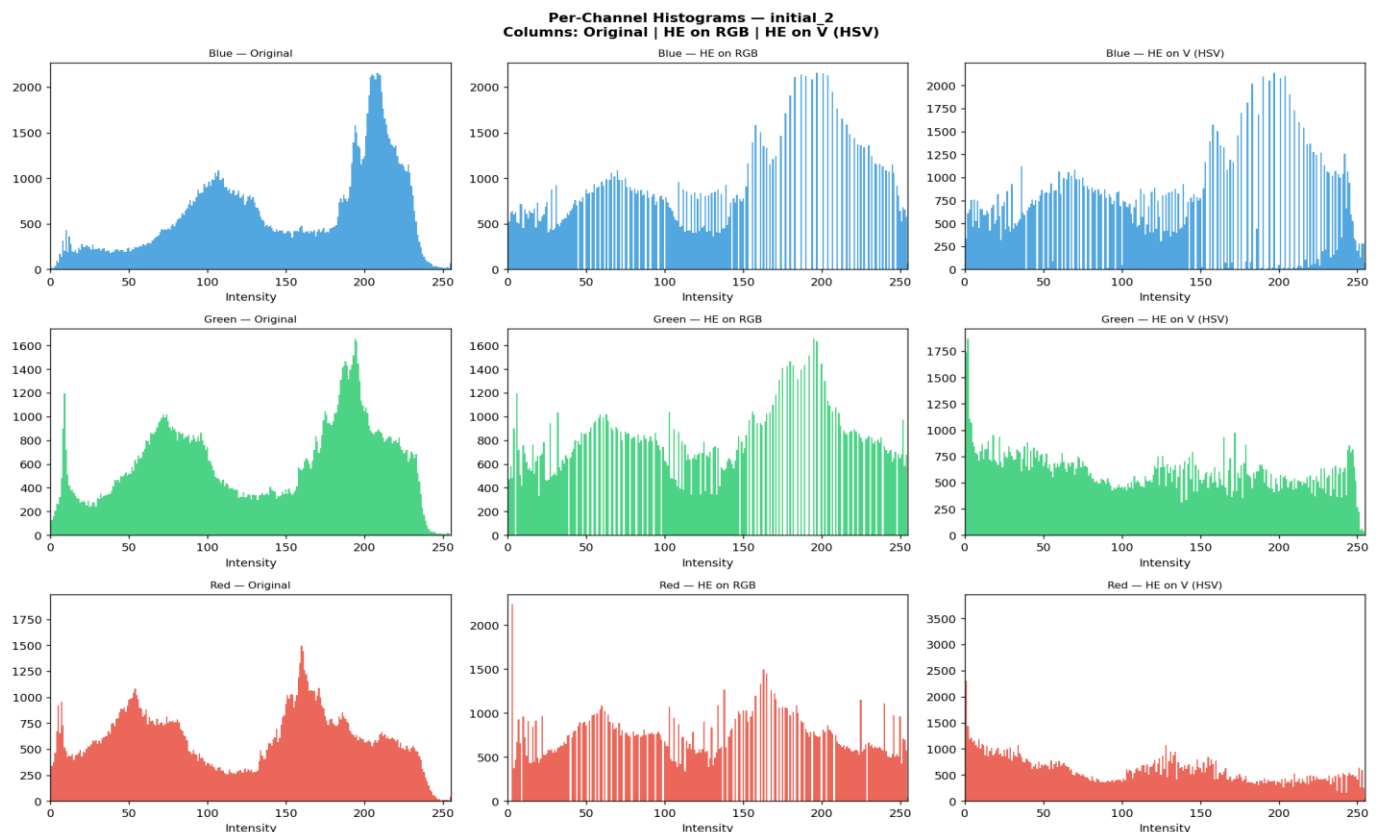


Figure 34: Per-channel histograms for original, per-channel HE, and V-channel HE outputs (initial_2.jpg). Each row corresponds to one channel (B, G, R). The V-channel HE produces more balanced per-channel distributions relative to per-channel HE, which independently flattens each channel.

Figure 35: Deconstructing the “Correct” (HSV) Method

This image strip shows the step-by-step process of the V-channel method, helping us understand how it works.

- Original: The starting image.
- Hue (H): This panel isolates the pure color component. The wild, psychedelic colors are just a way to visualize the different base hues in the image. Blue represents one hue, orange/red another, and so on. This part of the image is kept and not changed.
- Saturation (S): This panel isolates the intensity of the color. Bright white areas mean the color is very pure and vibrant, while dark/grey areas mean the color is more muted or washed out. This part is also kept and not changed.
- Value (V) (original): This panel isolates the brightness. It looks like a simple black-and-white version of the original photo, and you can see it has the same low-contrast, faded appearance.
- Value (V) (equalized): This is the key step. This is the brightness channel after histogram equalization was applied to it. Compared to the original V channel, the contrast is now dramatically higher. This high-contrast brightness information is then recombined with the original Hue and Saturation to create the final, improved image.

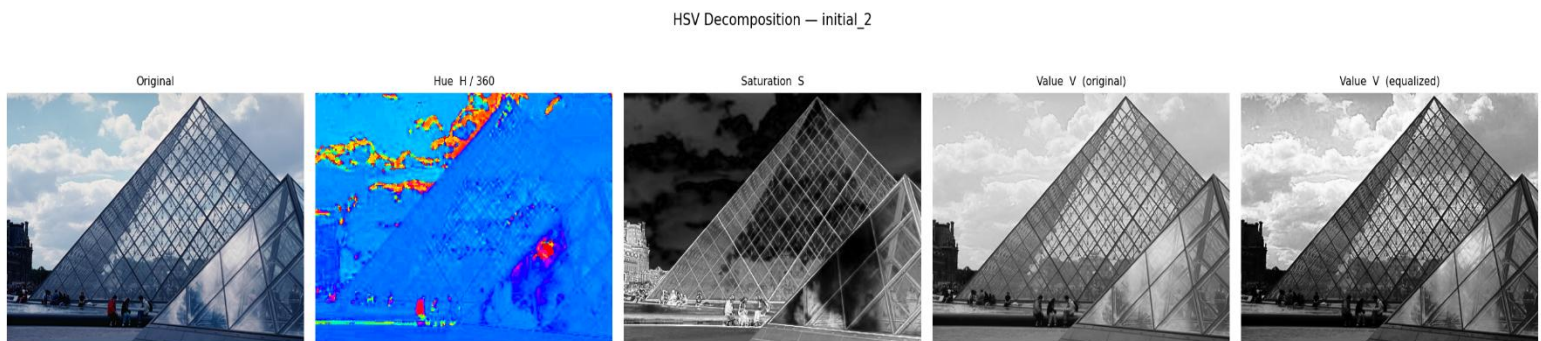


Figure 35: HSV decomposition of initial_2.jpg: original, Hue (H/360), Saturation (S), original Value (V), and equalised Value (V_eq). Only the Value channel is modified; Hue and Saturation channels are identical before and after.

4. Discussion

4.1 Interpolation Quality and Trade-offs

The quantitative results in Tables 1 and 2 reveal several consistent trends across all test images. Nearest-Neighbor interpolation consistently yields the lowest PSNR, as expected: the method introduces high-frequency block artifacts that manifest as large per-pixel errors.

Bilinear interpolation substantially outperforms nearest-Neighbor—the improvement is most dramatic in spectrally smooth images (peppers: +5.9 dB at $N=2$) but smaller in high-frequency imagery (mandrill: +2.3 dB at $N=2$).

The bicubic kernel offers marginal additional gains over bilinear for smooth content (peppers: +0.6 dB at $N=2$) but may slightly underperform bilinear for highly textured images such as the mandrill, where the negative sidelobes of the cubic kernel introduce mild ringing artifacts.

As the downsampling factor increases from $N=2$ to $N=8$, PSNR degradation is substantial for all methods, reflecting the irreversible loss of high-frequency information. The degradation is sharpest for nearest-Neighbor (cameraman: 22.4 $\rightarrow$ 15.8 dB, a drop of 6.6 dB) and more gradual for the smooth-kernel methods (bilinear cameraman: 25.5 $\rightarrow$ 19.1 dB, drop of 6.4 dB).

These results confirm that no interpolation method can fully recover information destroyed by aggressive downsampling.

4.2 Intensity Transformation Analysis

Log Transform in Fourier Images

In these special frequency images, the center is so incredibly bright that it completely blinds you to the delicate, faint details everywhere else. A logarithmic transformation acts like a visual equalizer: it severely dims the ultra-bright center while boosting the faint areas, compressing the huge range of brightness so you can see all the details at once.

Gamma Correction

Gamma correction is a smooth, predictable tool for fixing photo exposure.

Using a gamma value less than 1 ($\gamma < 1$) bends the brightness values upward, brightening up a dark image.

Using a value greater than 1 ($\gamma > 1$) bends them downward, darkening an overexposed image.

Bit-Plane Slicing

Every pixel's brightness is built from 8 binary layers (bits).

The Top 4 layers (Most Significant Bits) are the heavy lifters. They hold the core structure of the picture, accounting for about 94% of the total brightness (240 out of 255 levels).

The Bottom 4 layers (Least Significant Bits) only hold the remaining 6%. They contain mostly tiny, almost invisible details and random visual noise.

This is why an image built from just the top 4 layers still looks very good, while an image built from the bottom 4 layers just looks like random static.

4.3 Global vs. Local Histogram Equalization

GHE is computationally trivial ($O(N^2)$ for an $N \times N$ image) and produces reliable global contrast enhancement. However, it applies a single monotonic mapping to the entire image, which means that locally dark or bright regions are transformed uniformly regardless of their local context. Consequently, fine local detail may be suppressed if its local contrast is overwhelmed by the global distribution.

LHE resolves this by adapting the equalization mapping to the local Neighborhood statistics. The sliding-window implementation (window size 15×15) reveals crater details in the moon image that GHE obscures. The computational cost, however, is substantially higher: the optimized sliding-column implementation operates in $O(H \cdot W \cdot \text{win})$, which for the 537×358 moon image and $\text{win}=15$ yields approximately 2.9 million operations, compared to approximately 192,000 for GHE. The principal drawback of LHE is noise amplification: in near-uniform regions (e.g., the lunar sky), small stochastic intensity variations are magnified by the local equalization, introducing a speckle-like artifact that is visibly more pronounced than in the GHE output.

4.4 Histogram Matching and Color-Space Considerations

Histogram Matching

This technique alters a source image so its brightness and contrast style matches a “target” image. It works very well, but the result is rarely 100% perfect. Because digital images only use 256 whole numbers for brightness (no fractions), the math sometimes forces two slightly different shades in the original image to merge into the exact same shade in the final image. These “collisions” cause tiny, unavoidable differences between the final image and the target style.

Enhancing Color Images (RGB vs. HSV)

If you try to increase contrast by stretching the Red, Green, and Blue (RGB) channels independently, you ruin the delicate “recipe” that creates specific colors, leaving the image with weird, unnatural color tints.

The smart way is to convert the image to HSV format, which separates the actual color (Hue and Saturation) from the lightness (Value). By only boosting the contrast of the lightness (Value) channel, you make the image punchier while keeping the original colors perfectly safe and natural.

5. Conclusion

This report has documented the complete from-scratch implementation and rigorous experimental evaluation of five categories of digital image processing algorithms: diamond-pattern matrix embedding, spatial resolution resampling with three interpolation kernels, four classical intensity point operations with bit-plane analysis, global and local histogram equalization, and histogram specification with color-space-aware processing.

The principal findings are as follows:

- (1) In spatial resampling, bicubic interpolation achieves the highest PSNR for smooth and moderately textured images, while bilinear offers comparable performance with lower computational overhead; nearest-Neighbor consistently underperforms but remains useful where computation cost is critical.
- (2) In point operations, the logarithmic transformation is the most effective tool for high-dynamic-range imagery, gamma correction provides controlled exposure adjustment, and bit-plane analysis confirms that the four MSBs carry over 94% of the image's structural information.
- (3) In histogram processing, GHE provides efficient global contrast enhancement but cannot adapt to local variation, while LHE achieves superior local detail enhancement at the cost of noise amplification and a fifteen-fold increase in computational complexity.
- (4) Finally, histogram specification successfully transfers statistical distributions between images, and operating exclusively on the HSV Value channel is the principled approach for color-preserving intensity enhancement.